Welcome to CSC331 Fall 2025 Data Structures

Professor Kevin Byron (kbyron@bmcc.cuny.edu)
Department of CIS, Room F-1030-I
Office hours: Mon 2pm-5pm (in-person and Zoom)
Office phone: 212-220-1490
Borough of Manhattan Community College/CUNY

CSC331 Fall 2025
Data Structures

Week 11:

AVL tree drills
C++ vector container sample
Recursive Mergesort using vector

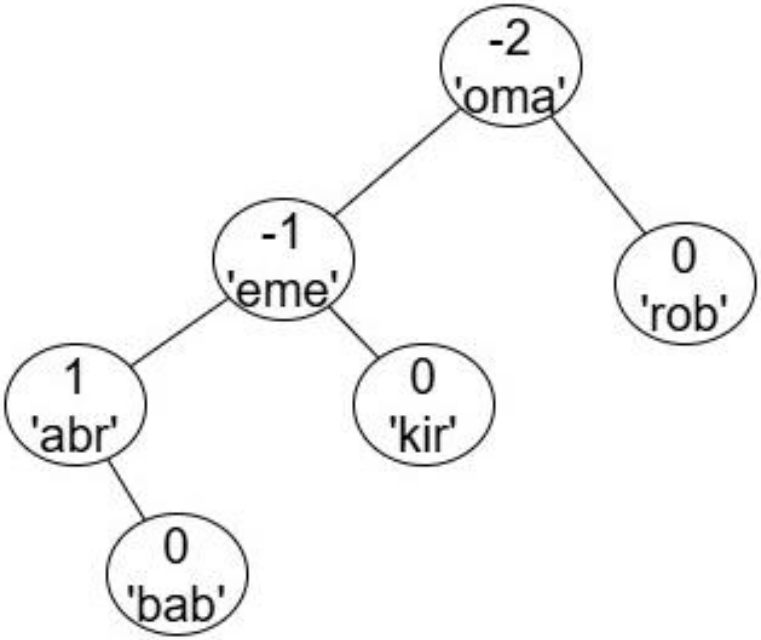
Professor Kevin Byron
Department of CIS
Borough of Manhattan Community College/CUNY

Reminders

- Program 4 due date
  - 1159pm Fri 11-21-2025
  - late submission receives no credit
- No class Thu 11-27-2025
- Program 5 due date
  - 1159pm Fri 12-12-2025
  - late submission receives no credit
- Final exam
  - Section 501 Tue 12-16-2025 – Zoom 520pm-735pm
  - Section 172 Wed 12-17-2025 – In person room M-1001 520pm-735pm
    - Closed book, closed notes, no devices
  - Section 500 Wed 12-17-2025 – In person room N-453 9am-1115am
    - Closed book, closed notes, no devices
- Posted on Brightspace
  - Week 11 lecture slides, C++ program to mergesort using vector (dm261.cpp), Test file for mergesort (dm258.dat), C++ Vector container sample program (chap4a.cpp)

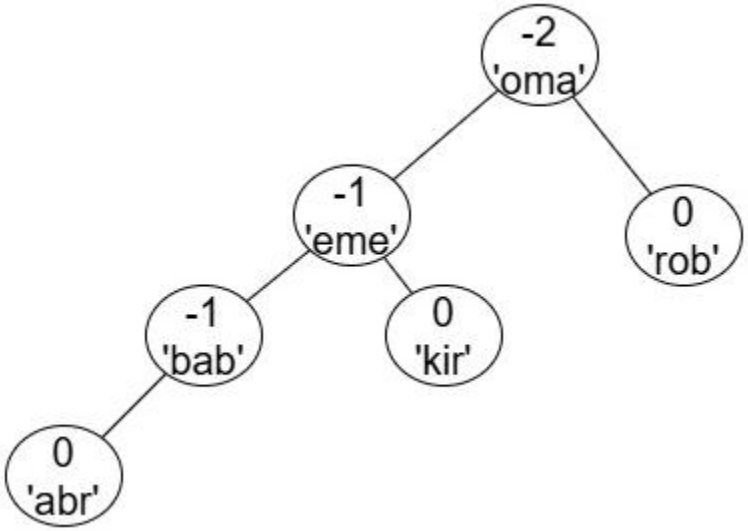
AVL tree drill 1

- A. Given the AVL tree below, choose the formula used to compute the ‘eme’ balance factor:
  - A. 0 minus 1
  - B. 1 minus 2
  - C. 2 minus 3
  - D. 3 minus 2
- B. Given the AVL tree below, choose the rotation tool needed to correct the imbalance, if any:
  - A. right rotate
  - B. left rotate
  - C. right-left rotate
  - D. left-right rotate
  - E. no correction needed



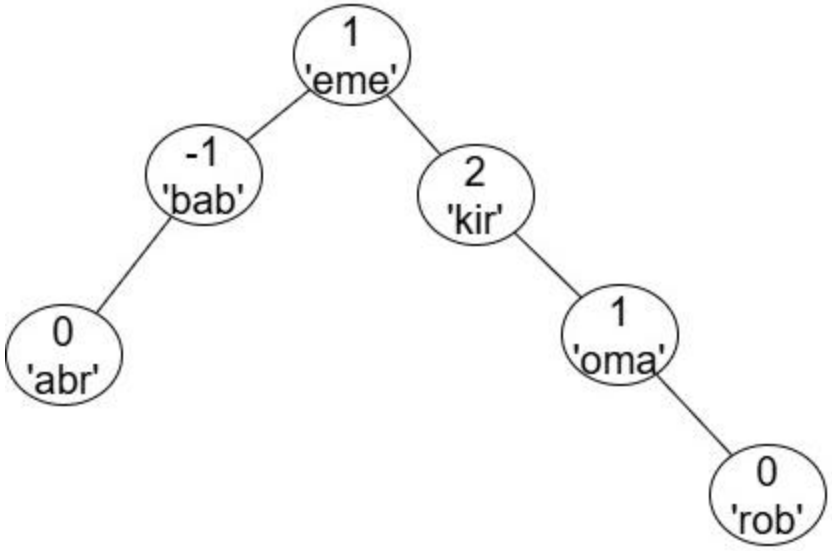
AVL tree drill 2

- A. Given the AVL tree below, choose the formula used to compute the ‘oma’ balance factor:
  - A. 0 minus 2
  - B. 1 minus 3
  - C. 2 minus 4
  - D. 3 minus 5
- B. Given the AVL tree below, choose the rotation tool needed to correct the imbalance, if any:
  - A. right rotate
  - B. left rotate
  - C. right-left rotate
  - D. left-right rotate
  - E. no correction needed



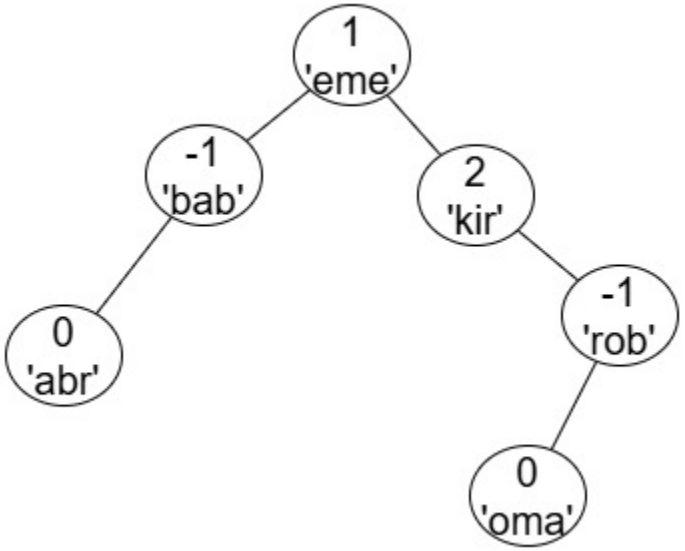
AVL tree drill 3

- A. Given the AVL tree below, choose the formula used to compute the ‘kir’ balance factor:
  - A. 2 minus 0
  - B. 3 minus 1
  - C. 3 minus 2
  - D. 4 minus 2
- B. Given the AVL tree below, choose the rotation tool needed to correct the imbalance, if any:
  - A. right rotate
  - B. left rotate
  - C. right-left rotate
  - D. left-right rotate
  - E. no correction needed



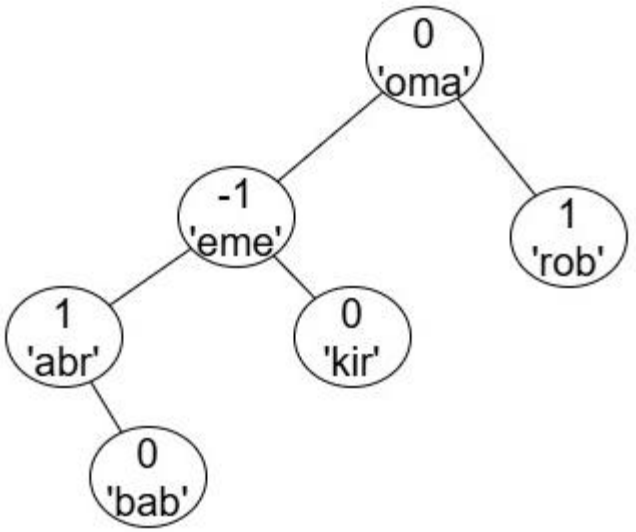
AVL tree drill 4

- A. Given the AVL tree below, choose the formula used to compute the ‘eme’ balance factor:
  - A. 1 minus 0
  - B. 2 minus 1
  - C. 3 minus 1
  - D. 3 minus 2
  - E. 4 minus 3
- B. Given the AVL tree below, choose the rotation tool needed to correct the imbalance, if any:
  - A. right rotate
  - B. left rotate
  - C. right-left rotate
  - D. left-right rotate
  - E. no correction needed



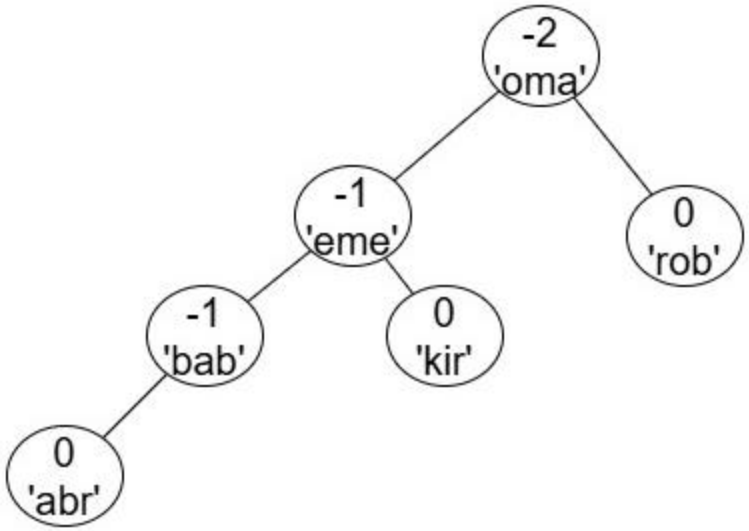
AVL tree drill 5

- Given the AVL tree below, choose the node or nodes having an incorrectly computed balance factor:
  - A. ‘abr’
  - B. ‘bab’
  - C. ‘eme’
  - D. ‘kir’
  - E. ‘oma’
  - F. ‘rob’
  - G. none of the above



AVL tree drill 6

- Given the AVL tree below, choose the node or nodes having an incorrectly computed balance factor:
  - A. ‘abr’
  - B. ‘bab’
  - C. ‘eme’
  - D. ‘kir’
  - E. ‘oma’
  - F. ‘rob’
  - G. none of the above



C++ vector container

- A Vector in C++ is a dynamic sequence container.
- A Vector is declared with a data type and variable name:
  - vector <int> rooms;
  - type defines a data type stored in a vector (e.g., <int>, <double> or <string>)
  - variable is a name that you choose for the data

C++ vector container

- The data elements are stored in contiguous storage. An iterator is used to access or move through the data. A modifier is used to update the vector.
- Iterator: an object that functions as a pointer
    - vector::begin() returns an iterator to point at the first element of a C++ vector.
    - vector::end() returns an iterator to point at past-the-end element of a C++ vector.

C++ vector container

- Modifier
  - vector::push_back() inserts elements onto the back.
  - vector::insert() inserts new elements at a specified location.
  - vector::pop_back() removes elements from the back.
  - vector::erase() removes a range of elements from a specified location.
    - myvector.erase (myvector.begin()+5);
    - myvector.erase (myvector.begin(),myvector.begin()+3);
  - vector::clear() removes all elements.

C++ vector container

```
//*************************************************************
// Author: D.S. Malik
//
// This program illustrates how to use a vector container in a
// program.
//*************************************************************
#include <iostream>                                          //Line 1
#include <vector>                                            //Line 2
using namespace std;                                         //Line 3
int main()                                                   //Line 4
```

C++ vector container

```
int main()                                          //Line 4
{                                                   //Line 5
  vector<int> intList;                              //Line 6
  intList.push_back(13);                            //Line 7
  intList.push_back(75);                            //Line 8
  intList.push_back(28);                            //Line 9
  intList.push_back(35);                            //Line 10
  cout << "Line 11: List Elements: ";               //Line 11
  for (int i = 0; i < 4; i++)                       //Line 12
    cout << intList[i] << " ";                      //Line 13
  cout << endl;                                     //Line 14
  for (int i = 0; i < 4; i++)                       //Line 15
    intList[i] *= 2;                                //Line 16
  cout << "Line 17: List Elements: ";               //Line 17
  for (int i = 0; i < 4; i++)                       //Line 18
    cout << intList[i] << " ";                      //Line 19
  cout << endl;                                     //Line 20
  vector<int>::iterator listIt;                     //Line 21
  cout << "Line 22: List Elements: ";               //Line 22
  for (listIt = intList.begin();
       listIt != intList.end(); ++listIt)           //Line 23
    cout << *listIt << " ";                         //Line 24
  cout << endl;                                     //Line 25
  listIt = intList.begin();                         //Line 26
  ++listIt;                                         //Line 27
  ++listIt;                                         //Line 28
  intList.insert(listIt, 88);                       //Line 29
  cout << "Line 30: List Elements: ";               //Line 30
  for (listIt = intList.begin();
       listIt != intList.end(); ++listIt)           //Line 31
    cout << *listIt << " ";                         //Line 32
  cout << endl;                                     //Line 33
  return 0;                                         //Line 34
}                                                   //Line 35
```

C++ vector container

```
C:\Users\Kevin Byron\work\cpp\fy20>.\chap4a
Line 11: List Elements: 13 75 28 35
Line 17: List Elements: 26 150 56 70
Line 22: List Elements: 26 150 56 70
Line 30: List Elements: 26 150 88 56 70

C:\Users\Kevin Byron\work\cpp\fy20>
```

Mergesort using vector

```
// dm261.cpp
// kbyron@bmcc.cuny.edu
// 11/26/19
// C++ program to recursively mergesort a vector of strings
// strings are read using redirected input from text file
#include <iostream>
#include <string>
#include <vector>
using namespace std;
```

Mergesort using vector

```
vector<string> MS(vector<string>list){
  if(list.size()==1)
    return list;
  else{
    vector<string> L1,L2,left,merged,right;
    int mid=list.size()/2;
    for(int i=0;i<mid;i++)
      left.push_back(list[i]);
    L1=MS(left);
    for(int i=mid;i<list.size();i++)
      right.push_back(list[i]);
    L2=MS(right);
    int L1i=0;
    int L2i=0;
    while(L1i<L1.size()&&L2i<L2.size()){
      if(L1[L1i] < L2[L2i]){
        merged.push_back(L1[L1i]);
        L1i++;
      }
      else{
        merged.push_back(L2[L2i]);
        L2i++;
      }
    }
    while(L1i<L1.size()){
      merged.push_back(L1[L1i]);
      L1i++;
    }
    while(L2i<L2.size()){
      merged.push_back(L2[L2i]);
      L2i++;
    }
    return merged;
  }
}
```

Mergesort using vector

```
int main () {
  string name;
  int count=0;
  vector<string> list;
  while (getline(cin,name)&&!cin.eof()){
      count++;
      list.push_back(name);
  }
  cout<< "the "<<count<<" names read as follows:\n";
  for (int i = 0; i < list.size(); i++)
    cout << i+1<<". "<<list[i] << "\n";
  vector<string> sortedlist=MS(list);
  cout<<endl<<"the "<<sortedlist.size()<<" names sorted as follows:\n";
  for (int i = 0; i < sortedlist.size(); i++)
    cout <<i+1<<". "<<sortedlist[i] << "\n";
}
```

Mergesort using vector

```
C:\Users\Kevin Byron\work\cpp\fy20>dm261 <dm258.dat
the 15 names read as follows:
1. Fatema
2. Howard
3. Marvin
4. Navid
5. Abdul
6. Justin
7. Saikat
8. Richie
9. Brittany
10. Fernando
11. Rayhan
12. Natalie
13. Tommy
14. Saul
15. Omaru
```

Mergesort using vector

```
the 15 names sorted as follows:
1. Abdul
2. Brittany
3. Fatema
4. Fernando
5. Howard
6. Justin
7. Marvin
8. Natalie
9. Navid
10. Omaru
11. Rayhan
12. Richie
13. Saikat
14. Saul
15. Tommy

C:\Users\Kevin Byron\work\cpp\fy20>
```

Assignment

- Read Malik chapter 4 – vector sequence containers
- Read Malik chapter 10 – sorting algorithms